Control de versiones con Git

Conceptos fundamentales

Que es el control de versiones

El control de versiones es un sistema que registra los cambios de un archivo o conjunto de archivos a lo largo del tiempo.

- Permite recuperar versiones anteriores de cualquier archivo
- Registra quien hizo cada cambio y cuando
- Facilita el trabajo en equipo sobre los mismos archivos
- Es la base del desarrollo de software colaborativo

Por que usar Git

Git es el sistema de control de versiones mas usado en el mundo. Es local, rapido y no necesita conexion para la mayoria de las operaciones.

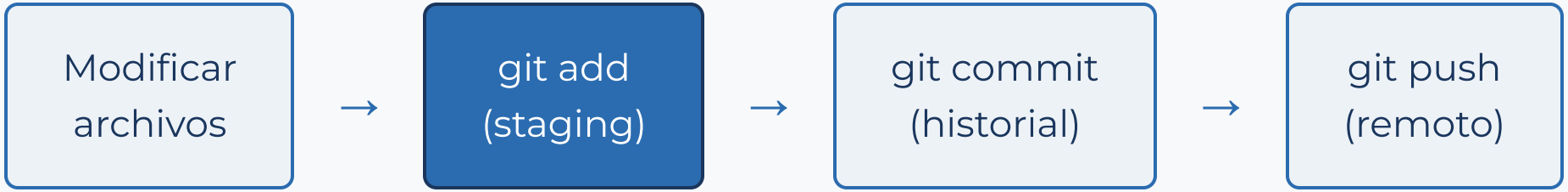
- Funciona offline: el historial vive en tu maquina
- Cada copia del repositorio es un backup completo
- Es el estandar de la industria desde hace mas de 15 anos

Los tres estados de Git

Todo archivo en un repositorio Git puede estar en uno de estos tres estados:

- **Modificado:** el archivo cambio pero Git todavia no sabe nada al respecto
- **Preparado (staged):** marcaste el cambio para incluirlo en el proximo commit
- **Confirmado (committed):** el cambio quedo registrado en el historial local

El flujo de trabajo basico



Cada `commit` es una fotografia del proyecto en un momento dado. El mensaje del commit describe que cambio y por que.

Comandos esenciales

```
git init
```

Inicializa un repositorio Git en la carpeta actual.

```
git add nombre_archivo.txt
```

Agrega un archivo al area de staging.

```
git commit -m "descripcion del cambio"
```

Confirma los cambios en el historial.

Ramas (branches)

Una rama es una linea de desarrollo independiente. Permite trabajar en una nueva funcionalidad sin afectar el codigo principal.

- La rama principal se llama convencionalmente `main` o `master`
- Crear una rama: `git branch nombre-rama`
- Cambiar a una rama: `git checkout nombre-rama`
- Fusionar una rama en `main`: `git merge nombre-rama`

Comparacion de flujos de trabajo

Escenario	Sin control de versiones	Con Git
Revertir un error	Imposible si no hay backup	`git revert` o `git checkout`
Trabajar en equipo	Archivos que se pisan entre si	Ramas y merges controlados
Auditar cambios	No hay registro	`git log` muestra el historial completo
Probar algo nuevo	Copiar carpeta manualmente	Crear una rama y experimentar

Como escribir un buen mensaje de commit

El mensaje de commit es el registro permanente de que cambio y por que. Un mensaje vago inutiliza el historial.

Mensaje poco util:

```
arreglos varios
```

Mensaje util:

```
Corregir calculo de descuento para usuarios con membresia anual
El descuento se aplicaba sobre el precio sin impuestos en lugar

del precio final. Esto generaba un cobro incorrecto en checkout.
```

Repositorios remotos

Un repositorio remoto es una copia del proyecto alojada en un servidor (GitHub, GitLab, Bitbucket, etc.). Sirve como punto de sincronizacion entre varios colaboradores.

- `git remote add origin URL`: conecta el repo local con el remoto
- `git push`: sube los commits locales al servidor
- `git pull`: baja y fusiona los cambios del servidor

Proximos pasos

Practicar es la unica forma de internalizar estos comandos.

Para empezar hoy:
Instalar Git, crear una carpeta de prueba y hacer los primeros tres commits de cualquier proyecto propio.

En la proxima clase:

- Resolucion de conflictos al fusionar ramas
- Uso de `.gitignore`
- Flujos de trabajo en equipo (pull requests)